National Higher School of Artificial Intelligence

Course: Introduction to Mobile Robotics

Lab Report 1: Foundations of ROS 2

Computational Graph, Workspace Architecture, C++ Implementation, and Telemetry

Abdelbari Tarek SI BACHIR

October 26, 2026

Abstract

This report describes the work completed for Lab 1 in the Mobile Robotics course. The lab introduced the ROS 2 Jazzy architecture and its programming model through command-line tools, the turtlesim simulator, workspace overlays, and a custom C++ publisher-subscriber package. Since the host workstation runs Arch Linux, the ROS 2 Jazzy environment was established inside an integrated Distrobox container using the official Ubuntu 24.04 LTS base.

1 Introduction

ROS 2 is a distributed framework for building robotics applications whose modular components communicate asynchronously and synchronously. It leverages DDS (Data Distribution Service) for peer-to-peer communication and node discovery, avoiding single-point-of-failure centralized masters. During this laboratory session, the core entities of the ROS 2 computational graph were explored:

- **Nodes:** Individual operational processes performing isolated computation.
- **Topics:** Asynchronous publish-subscribe communication buses carrying typed message payloads.
- **Services:** Synchronous request-and-response interfaces for discrete operations.
- **Actions:** Asynchronous goal-oriented execution pipelines supporting preemption, status reporting, and periodic feedback.
- **Parameters:** Node-level configuration variables dynamically queryable and reconfigurable at runtime.

2 Execution Environment

As the host machine runs an Arch Linux rolling release, ROS 2 Jazzy was isolated within a dedicated **Distrobox** container initialized from the official `osrf/ros:jazzy-desktop` image. Distrobox provides native integration with the host X11/Wayland display server, Direct Rendering Infrastructure (DRI) GPU acceleration, and bidirectional access to the host workspace without container-boundary permission overhead.

3 Exploring the ROS 2 Graph

3.1 Nodes and Remapping

The simulation was initialized by running the `turtlesim_node` executable and the interactive keyboard teleoperation node `turtle_teleop_key`.

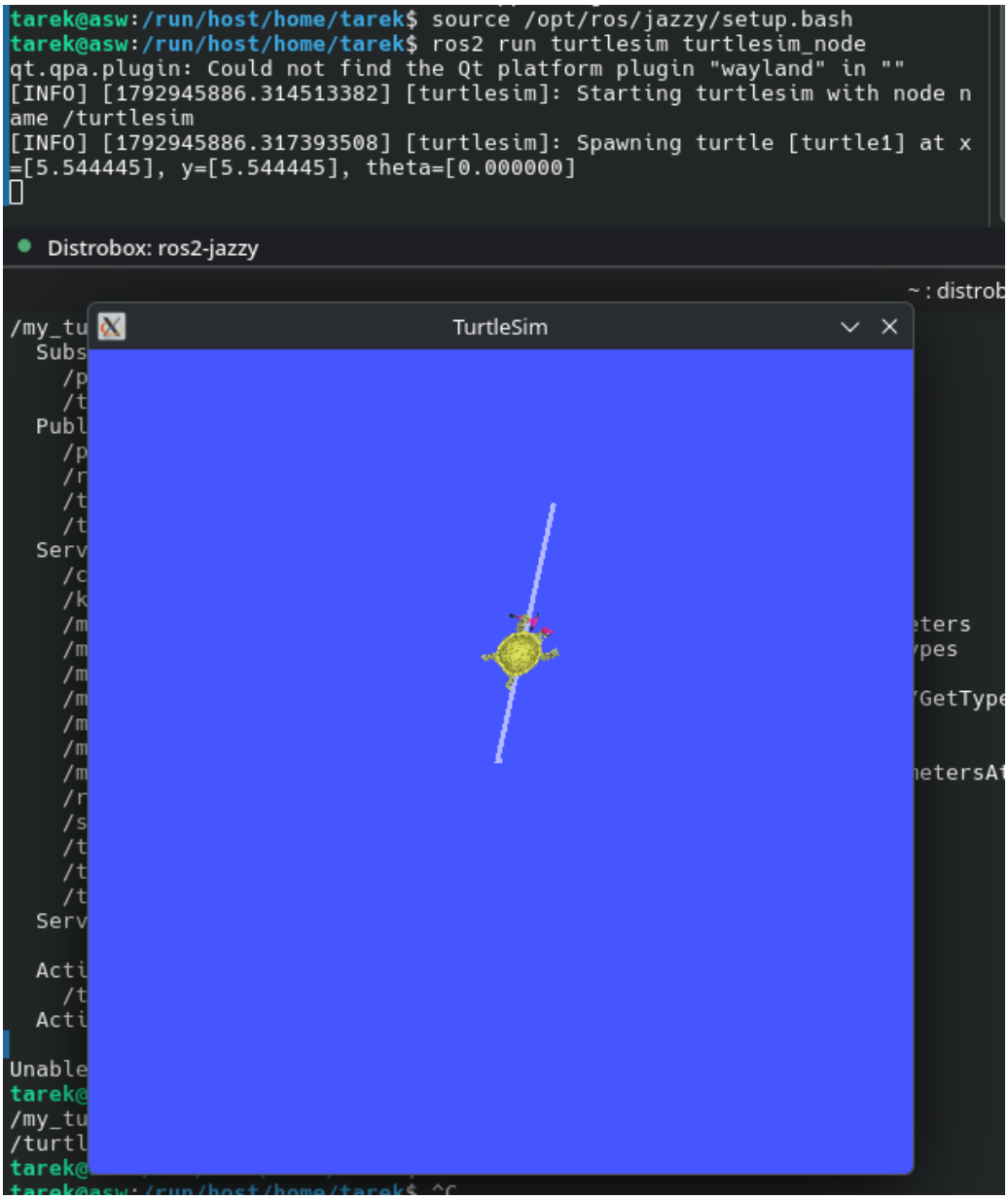


Figure 1: Initialization of the graphical `turtlesim_node` simulator viewport.

Active nodes registered in the computation graph were queried using `ros2 node list`. Node names were dynamically remapped at startup without altering compiled binaries:

```
ros2 run turtlesim turtlesim_node --ros-args --remap __node:=my_turtle
```

Node introspection using `ros2 node info` allowed structural analysis of endpoints for both the simulator and controller:

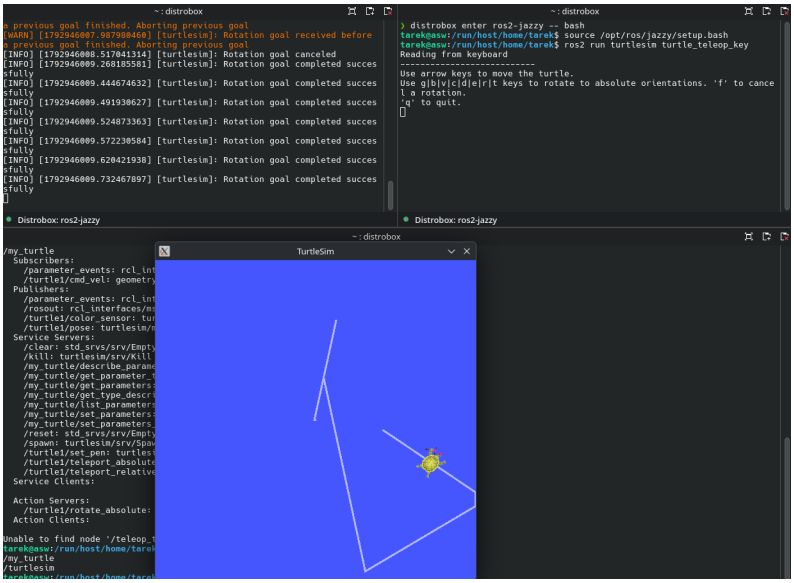


Figure 2: Execution of `turtle_teleop_key` for terminal keyboard control.

```
tarek@asw:/run/host/home/tarek$ ros2 node list
WARNING: Be aware that there are nodes in the graph that share an exact name, which can have unintended side effects.
/my_turtle
/teleop_turtle
/teleop_turtle
/turtlesim
/turtlesim
```

Figure 3: Output of `ros2 node list` displaying active graph nodes.

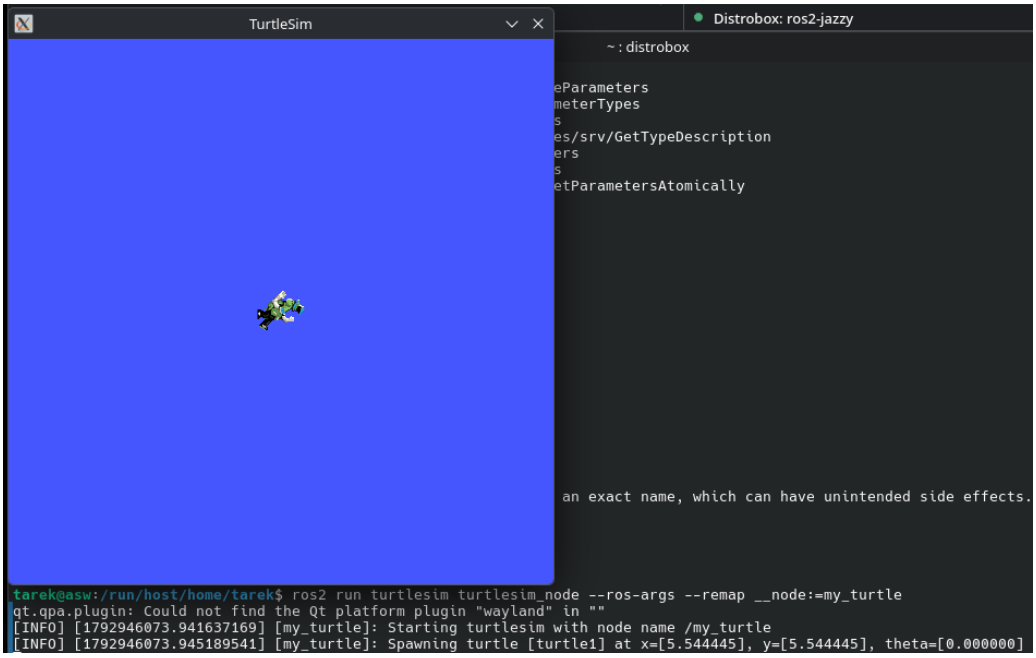


Figure 4: Remapping node name from `/turtlesim` to `/my_turtle` at launch.

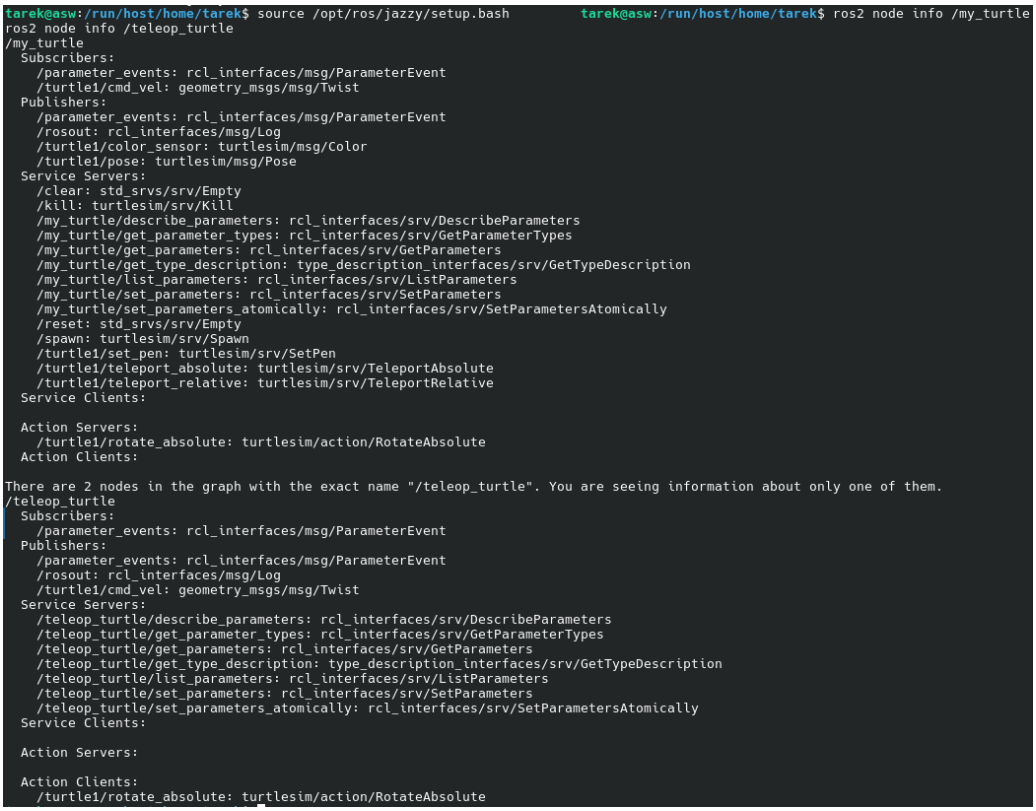


Figure 5: Detailed node interface inspection comparing publishers, subscribers, services, and actions.

3.2 Topic Communication and Terminal Publishing

Inspecting `/turtle1/cmd_vel` revealed that it operates on the `geometry_msgs/msg/Twist` interface:

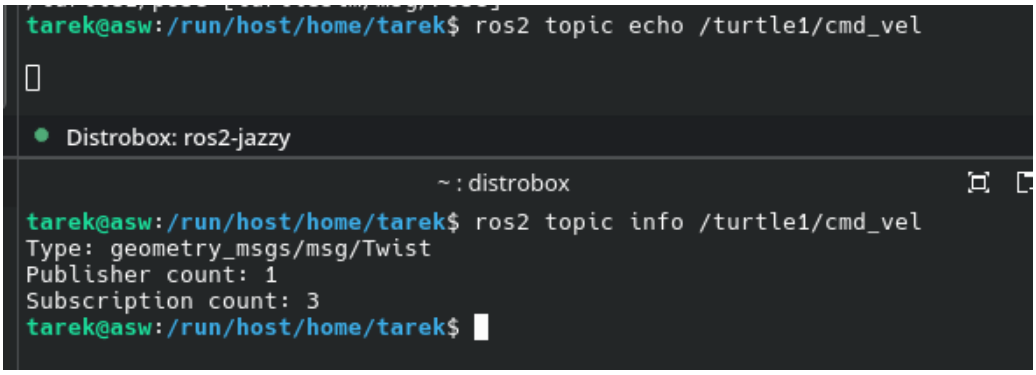


Figure 6: Introspection of `/turtle1/cmd_vel` topic type and publisher/subscriber count.

Messages were injected into the topic directly from the command line:
Continuous steady-state velocity was generated at 1 Hz to command circular trajectories:

```
ros2 topic pub --rate 1 /turtle1/cmd_vel geometry_msgs/msg/Twist \
  "{linear: {x: 2.0, y: 0.0, z: 0.0}, angular: {x: 0.0, y: 0.0, z: 1.8}}
   "
```

The transmission frequency of state telemetry was verified:

```
tarek@asw:/run/host/home/tarek$ ros2 interface show geometry_msgs/msg/Twist
# This expresses velocity in free space broken into its linear and angular parts.

Vector3  linear
        float64 x
        float64 y
        float64 z
Vector3  angular
        float64 x
        float64 y
        float64 z
tarek@asw:/run/host/home/tarek$
```

Figure 7: Interface definition showing 3D linear and angular vector primitives.

```
---
linear:
  x: 2.0
  y: 0.0
  z: 0.0
angular:
  x: 0.0
  y: 0.0
  z: 1.8
---
```

```
Distrobox: ros2-jazzy
~ : distrobox
tarek@asw:/run/host/home/tarek$ ros2 topic pub --once /turtle1/cmd_vel geometry_msg
s/msg/Twist "{linear: {x: 2.0, y: 0.0, z: 0.0}, angular: {x: 0.0, y: 0.0, z: 1.8}}"
publisher: beginning loop
publishing #1: geometry_msgs.msg.Twist(linear=geometry_msgs.msg.Vector3(x=2.0, y=0.
0, z=0.0), angular=geometry_msgs.msg.Vector3(x=0.0, y=0.0, z=1.8))

tarek@asw:/run/host/home/tarek$
```

Figure 8: Publishing a single manual velocity vector using `ros2 topic pub -once`.

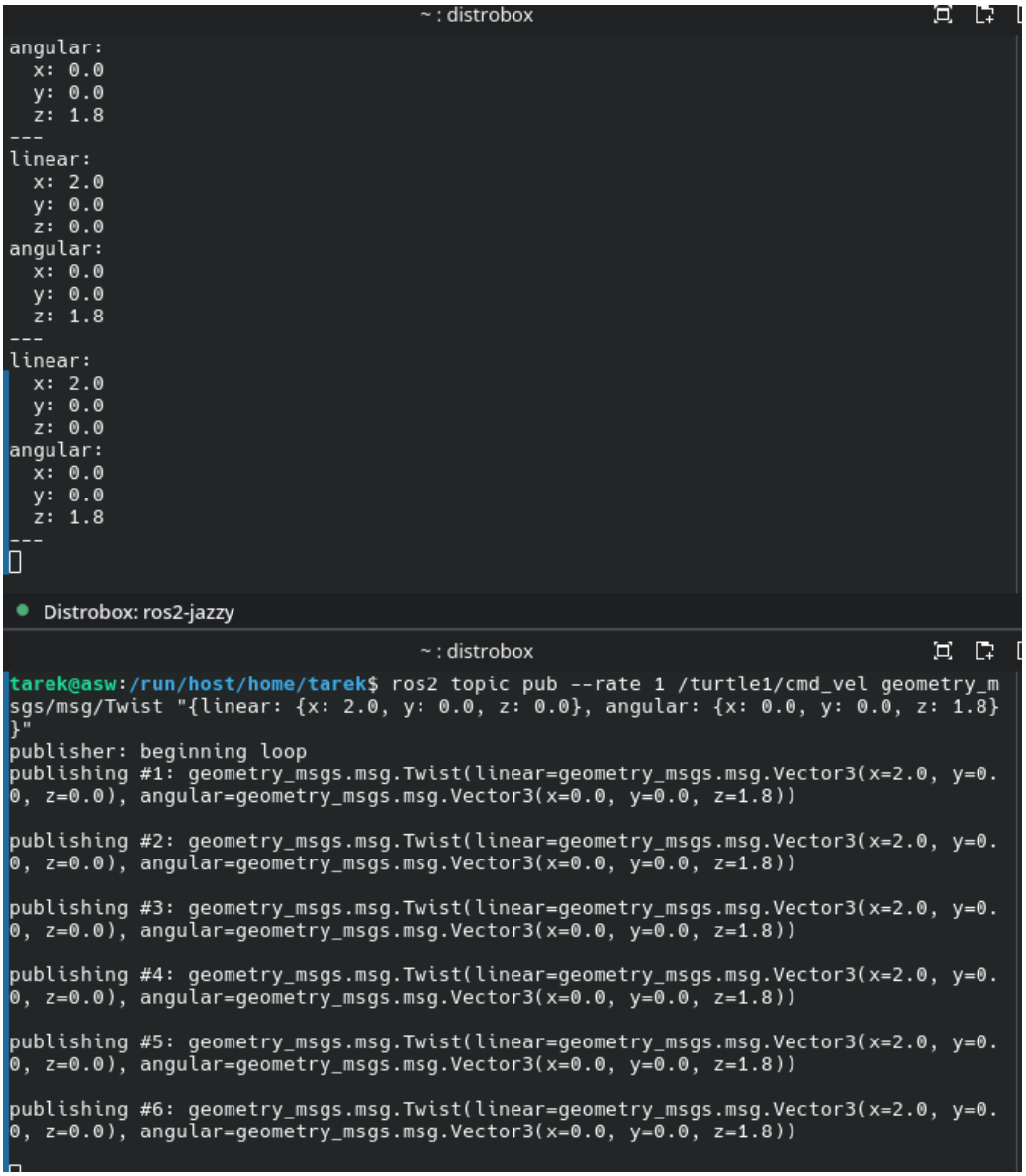


Figure 9: Continuous 1 Hz velocity publishing commanding periodic circular motion.

```
^Ctarek@asw:/run/host/home/tarek$ ros2 topic hz /turtle1/pose
average rate: 124.583
        min: 0.005s max: 0.011s std dev: 0.00247s window: 127
average rate: 124.818
        min: 0.004s max: 0.011s std dev: 0.00248s window: 253
average rate: 124.902
        min: 0.004s max: 0.011s std dev: 0.00247s window: 379
average rate: 124.899
        min: 0.004s max: 0.011s std dev: 0.00245s window: 505
average rate: 124.914
        min: 0.002s max: 0.014s std dev: 0.00248s window: 631
average rate: 124.948
        min: 0.002s max: 0.014s std dev: 0.00248s window: 757
average rate: 124.951
        min: 0.002s max: 0.014s std dev: 0.00247s window: 883
average rate: 124.961
        min: 0.002s max: 0.014s std dev: 0.00247s window: 1009
average rate: 124.960
        min: 0.002s max: 0.014s std dev: 0.00246s window: 1135
average rate: 124.967
        min: 0.002s max: 0.014s std dev: 0.00246s window: 1261
average rate: 124.971
        min: 0.002s max: 0.014s std dev: 0.00247s window: 1387
average rate: 124.972
        min: 0.002s max: 0.014s std dev: 0.00247s window: 1513
average rate: 124.976
        min: 0.002s max: 0.014s std dev: 0.00246s window: 1639
average rate: 124.975
        min: 0.002s max: 0.014s std dev: 0.00246s window: 1765
```

Figure 10: Evaluating update frequency of `/turtle1/pose` ($\sim$ 62.5 Hz).

3.3 Services, Parameters, and Actions

Active service endpoints were identified and inspected:

Parameters were queried, read, and dynamically modified at runtime:

Action goal dispatching with progressive feedback was executed using the action CLI:

4 Workspace Layering: Underlays and Overlays

The core ROS 2 installation at `/opt/ros/jazzy` served as the *underlay*, while `~/ros2_ws` acted as the development *overlay*.

Following source modifications to `turtle_frame.cpp` to retitle the window to `"MyTurtleSim"`, the overlay was compiled. Sourcing `~/ros2_ws/install/setup.bash` successfully gave precedence to the local overlay build over the global system underlay.

5 Implementation of Custom C++ Package (`cpp_pubsub`)

A custom package `cpp_pubsub` was created using the `ament_cmake` build system:

The publisher node (`talker`) periodically dispatched serialized `std_msgs/msg/String` payloads, while the subscriber node (`listener`) processed incoming messages via bound member callbacks.

The build dependencies and executable targets were defined in `package.xml` and `CMakeLists.txt`:

6 Launch Files, Rosbags, and Diagnostics

6.1 Multi-Node Launch Orchestration

Multiple isolated simulator instances were brought up simultaneously using a launch file:

```
ros2 launch turtlesim multisim.launch.py
```

Independent control was demonstrated by addressing distinct namespaced topics:

```
tarek@asw:/run/host/home/tarek$ ros2 service liststst
ros2 service list -t
/clear
/kill
/my_turtle/describe_parameters
/my_turtle/get_parameter_types
/my_turtle/get_parameters
/my_turtle/get_type_description
/my_turtle/list_parameters
/my_turtle/set_parameters
/my_turtle/set_parameters_atomically
/reset
/spawn
/turtle1/set_pen
/turtle1/teleport_absolute
/turtle1/teleport_relative
/turtlesim/describe_parameters
/turtlesim/get_parameter_types
/turtlesim/get_parameters
/turtlesim/get_type_description
/turtlesim/list_parameters
/turtlesim/set_parameters
/turtlesim/set_parameters_atomically
/clear [std_srvs/srv/Empty]
/kill [turtlesim/srv/Kill]
/my_turtle/describe_parameters [rcl_interfaces/srv/DescribeParameters]
/my_turtle/get_parameter_types [rcl_interfaces/srv/GetParameterTypes]
/my_turtle/get_parameters [rcl_interfaces/srv/GetParameters]
/my_turtle/get_type_description [type_description_interfaces/srv/GetTypeDescription
]
/my_turtle/list_parameters [rcl_interfaces/srv/ListParameters]
/my_turtle/set_parameters [rcl_interfaces/srv/SetParameters]
/my_turtle/set_parameters_atomically [rcl_interfaces/srv/SetParametersAtomically]
/reset [std_srvs/srv/Empty]
/spawn [turtlesim/srv/Spawn]
/turtle1/set_pen [turtlesim/srv/SetPen]
/turtle1/teleport_absolute [turtlesim/srv/TeleportAbsolute]
/turtle1/teleport_relative [turtlesim/srv/TeleportRelative]
/turtlesim/describe_parameters [rcl_interfaces/srv/DescribeParameters]
/turtlesim/get_parameter_types [rcl_interfaces/srv/GetParameterTypes]
/turtlesim/get_parameters [rcl_interfaces/srv/GetParameters]
/turtlesim/get_type_description [type_description_interfaces/srv/GetTypeDescription
]
/turtlesim/list_parameters [rcl_interfaces/srv/ListParameters]
/turtlesim/set_parameters [rcl_interfaces/srv/SetParameters]
/turtlesim/set_parameters_atomically [rcl_interfaces/srv/SetParametersAtomically]
```

Figure 11: Listing all service servers and associated service types.

```
tarek@asw:/run/host/home/tarek$ ros2 service type /clear
std_srvs/srv/Empty
tarek@asw:/run/host/home/tarek$
```

Figure 12: Querying the specific service type of `/clear`.

```
tarek@asw:/run/host/home/tarek$ ros2 service find std_srvs/srv/Empty
/clear
/reset
tarek@asw:/run/host/home/tarek$
```

Figure 13: Locating all service instances utilizing the `std_srvs/srv/Empty` type.

```
tarek@asw:/run/host/home/tarek$ ros2 interface show turtlesim/srv/Spawn
float32 x
float32 y
float32 theta
string name # Optional.  A unique name will be created and returned if this is empt
y
---
string name
```

Figure 14: Definition of `turtlesim/srv/Spawn` request and response parameters.

```
tarek@asw:/run/host/home/tarek$ ros2 service call /spawn turtlesim/srv/Spawn "{x: 2
.0, y: 2.0, theta: 0.2, name: 'turtle2'}"
waiting for service to become available...
requester: making request: turtlesim.srv.Spawn_Request(x=2.0, y=2.0, theta=0.2, nam
e='turtle2')

response:
turtlesim.srv.Spawn_Response(name='turtle2')

tarek@asw:/run/host/home/tarek$
```

Figure 15: Service execution: Spawning a secondary turtle (`turtle2`) at $(x = 3.0, y = 3.0)$.

```
tarek@asw:/run/host/home/tarek$ ros2 service call /clear std_srvs/srv/Empty
waiting for service to become available...
requester: making request: std_srvs.srv.Empty_Request()

response:
std_srvs.srv.Empty_Response()

tarek@asw:/run/host/home/tarek$
```

Figure 16: Calling the `/clear` service to erase accumulated background drawing tracks.

```
tarek@asw:/run/host/home/tarek$ ros2 param list
/my_turtle:
  background_b
  background_g
  background_r
  holonomic
  qos_overrides./parameter_events.publisher.depth
  qos_overrides./parameter_events.publisher.durability
  qos_overrides./parameter_events.publisher.history
  qos_overrides./parameter_events.publisher.reliability
  start_type_description_service
  use_sim_time
/turtlesim:
  background_b
  background_g
  background_r
  holonomic
  qos_overrides./parameter_events.publisher.depth
  qos_overrides./parameter_events.publisher.durability
  qos_overrides./parameter_events.publisher.history
  qos_overrides./parameter_events.publisher.reliability
  start_type_description_service
  use_sim_time
tarek@asw:/run/host/home/tarek$
```

Figure 17: Listing runtime parameters hosted by the simulator node.

```
tarek@asw:/run/host/home/tarek$ ros2 param get /turtlesim background_g
Integer value is: 86
tarek@asw:/run/host/home/tarek$
```

Figure 18: Reading the background green channel parameter value.

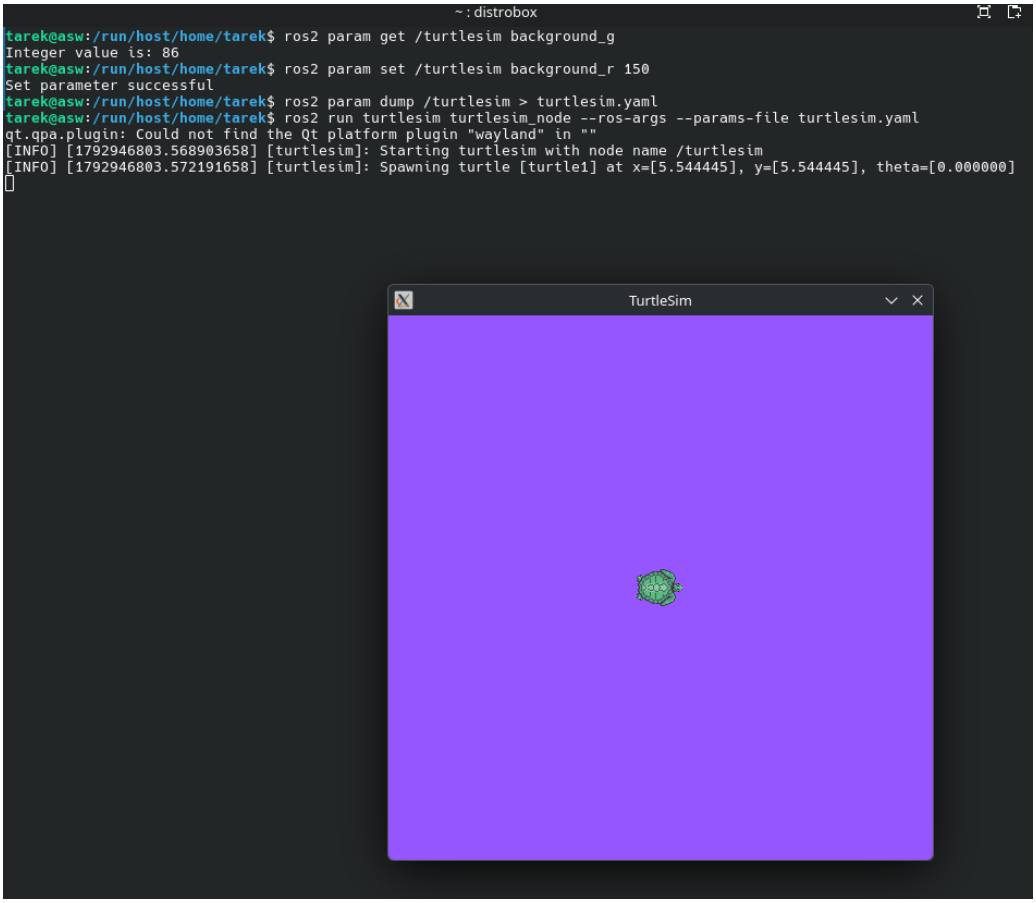


Figure 19: Dynamically updating `background_r` to modify window background color.

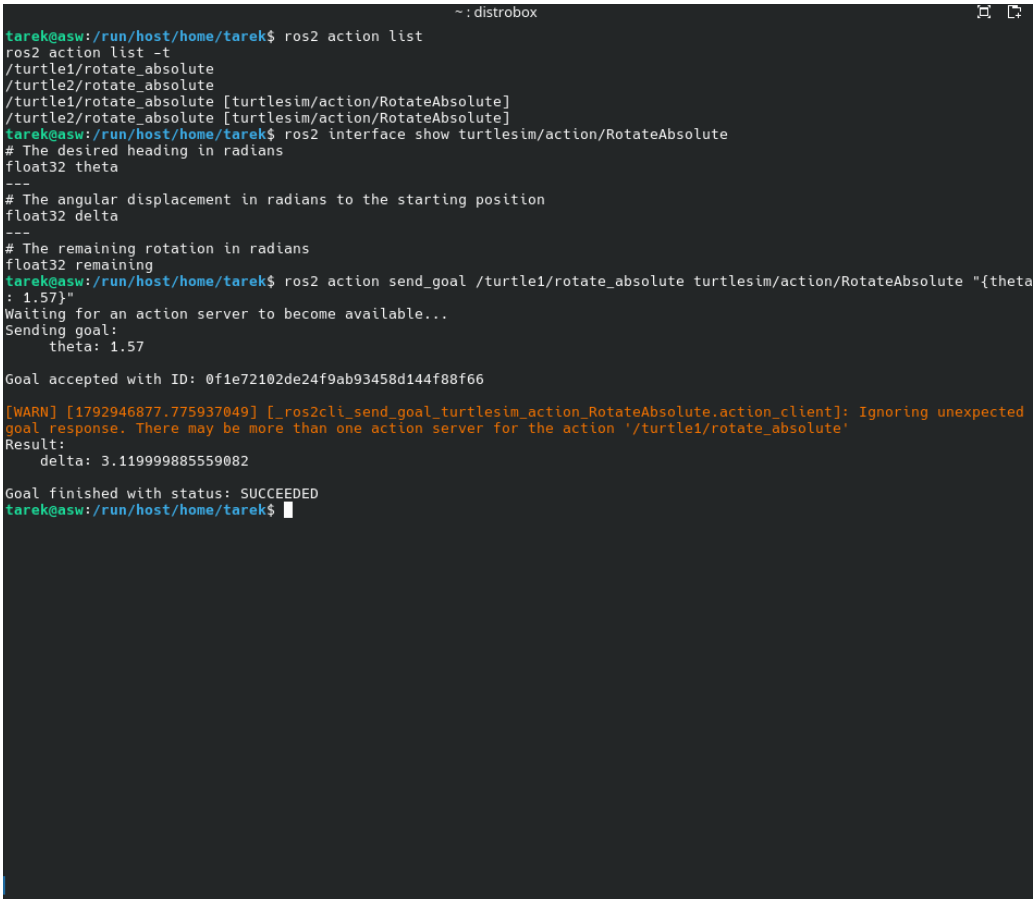


Figure 20: Structure of the `RotateAbsolute` action interface: Goal, Result, and Feedback.

```
Feedback:
    remaining: 0.14675581455230713

Feedback:
    remaining: 0.12432646751403809

Feedback:
    remaining: 0.130755901336669992

Feedback:
    remaining: 0.10832643508911133

Feedback:
    remaining: 0.11475586891174316

Feedback:
    remaining: 0.09232640266418457

Feedback:
    remaining: 0.0987558364868164

Feedback:
    remaining: 0.08275580406188965

Feedback:
    remaining: 0.07632648944854736

Feedback:
    remaining: 0.060326457023620605

Feedback:
    remaining: 0.06675589084625244

Feedback:
    remaining: 0.04432642459869385

Feedback:
    remaining: 0.050755858421325684

Feedback:
    remaining: 0.02832639217376709

Feedback:
    remaining: 0.034755825996398926

Feedback:
    remaining: 0.012326478958129883

Result:
    delta: -3.119999885559082
Goal finished with status: SUCCEEDED
tarek@asw:/run/host/home/tarek$
```

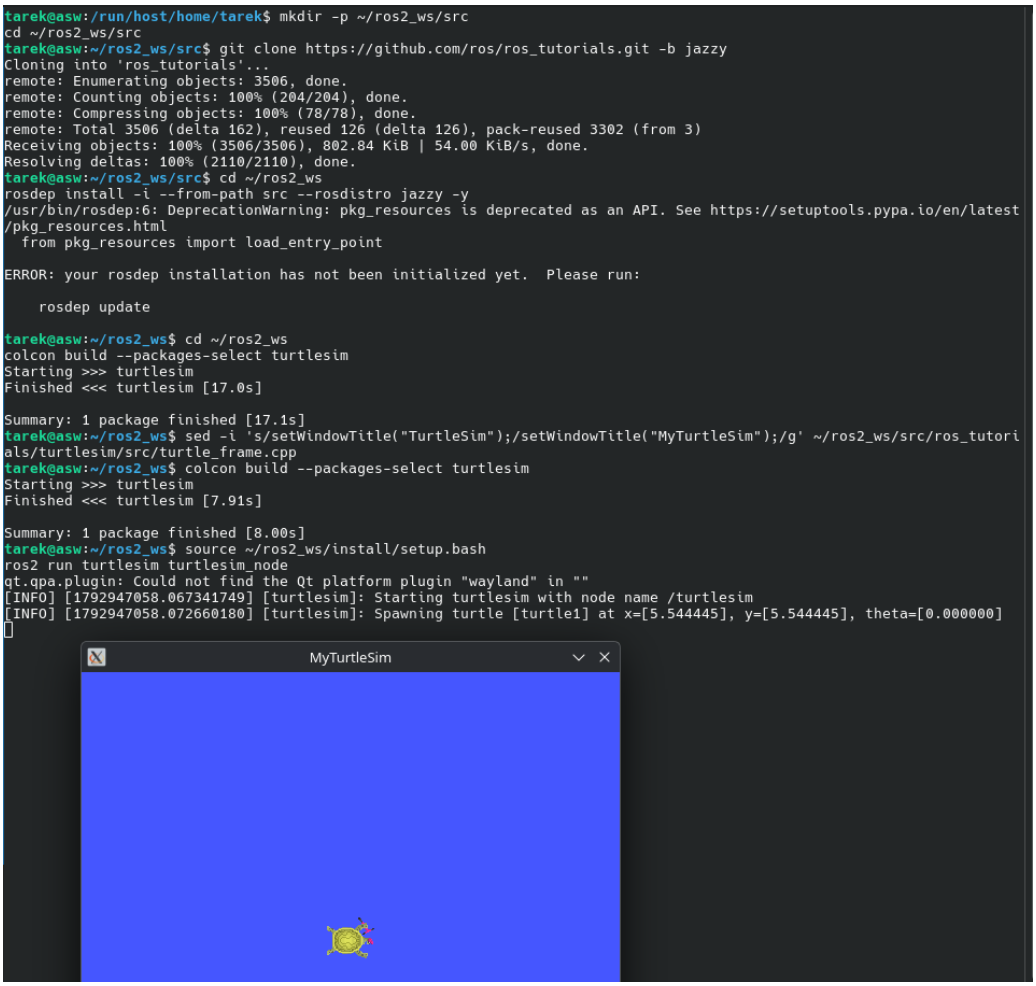


Figure 22: Cloning the `ros_tutorials` repository into the overlay workspace source tree.

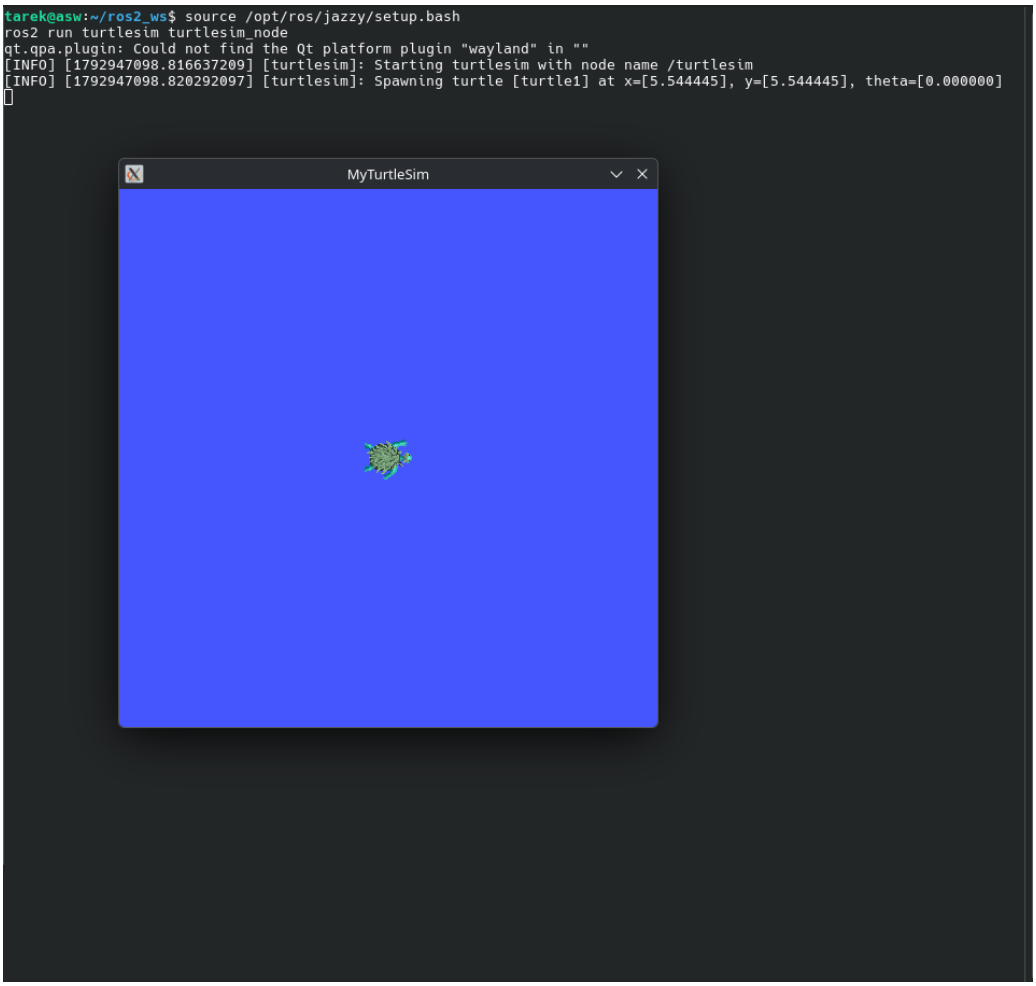


Figure 23: Resolving package dependencies and compiling the modified overlay with `colcon`.

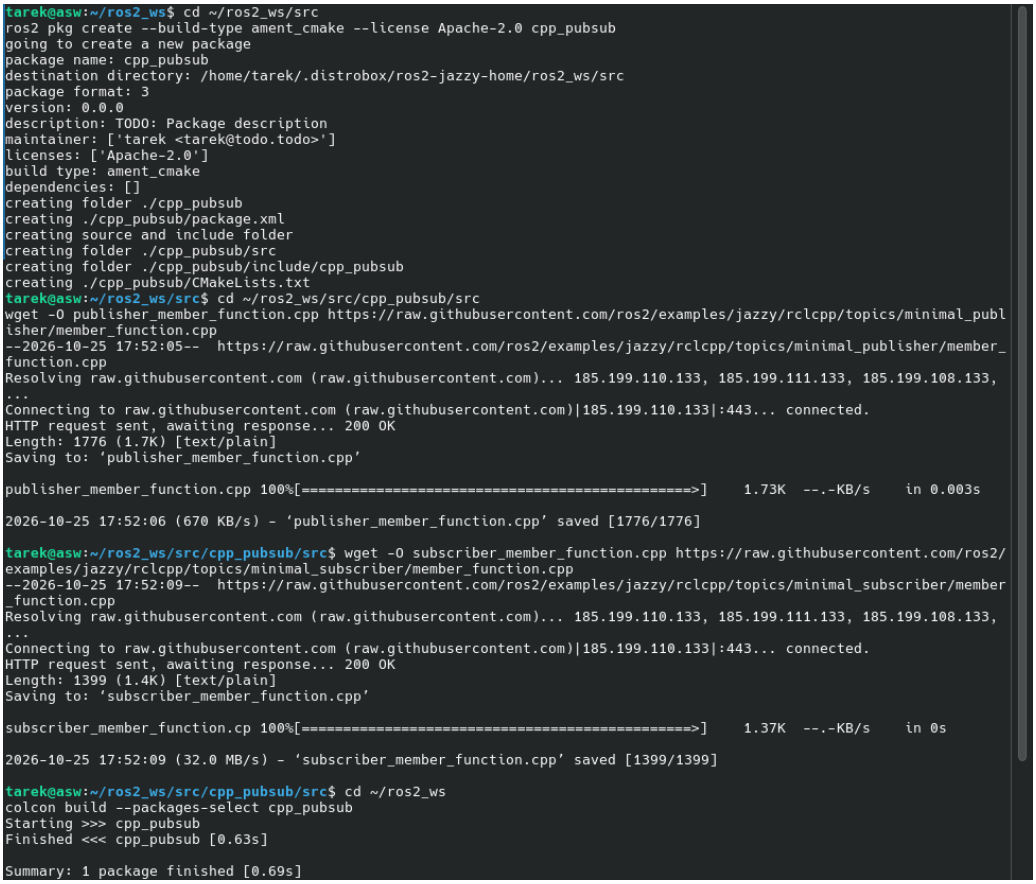


Figure 24: Creating the `cpp_pubsub` package skeleton with `ros2 pkg create`.

```
tarek@asw:/run/host/home/tarek$ distrobox enter ros2-jazzy -- bashsh
source ~/ros2_ws/install/setup.bash
ros2 run cpp_pubsub listener
bash: distrobox: command not found
[INFO] [1792947275.111481863] [minimal_subscriber]: I heard: 'Hello, world! 52'
[INFO] [1792947275.611710149] [minimal_subscriber]: I heard: 'Hello, world! 53'
[INFO] [1792947276.111547634] [minimal_subscriber]: I heard: 'Hello, world! 54'
[INFO] [1792947276.611563559] [minimal_subscriber]: I heard: 'Hello, world! 55'
[INFO] [1792947277.111721817] [minimal_subscriber]: I heard: 'Hello, world! 56'
[INFO] [1792947277.611290725] [minimal_subscriber]: I heard: 'Hello, world! 57'
[INFO] [1792947278.111800672] [minimal_subscriber]: I heard: 'Hello, world! 58'
[INFO] [1792947278.611700356] [minimal_subscriber]: I heard: 'Hello, world! 59'
[INFO] [1792947279.111715595] [minimal_subscriber]: I heard: 'Hello, world! 60'
Distrobox: ros2-jazzy
~ : distrobox
[INFO] [1792947263.111324204] [minimal_publisher]: Publishing: 'Hello, world! 28'
[INFO] [1792947263.611218185] [minimal_publisher]: Publishing: 'Hello, world! 29'
[INFO] [1792947264.111216782] [minimal_publisher]: Publishing: 'Hello, world! 30'
[INFO] [1792947264.611223745] [minimal_publisher]: Publishing: 'Hello, world! 31'
[INFO] [1792947265.111132229] [minimal_publisher]: Publishing: 'Hello, world! 32'
[INFO] [1792947265.611128219] [minimal_publisher]: Publishing: 'Hello, world! 33'
[INFO] [1792947266.111108043] [minimal_publisher]: Publishing: 'Hello, world! 34'
[INFO] [1792947266.611199747] [minimal_publisher]: Publishing: 'Hello, world! 35'
[INFO] [1792947267.111235567] [minimal_publisher]: Publishing: 'Hello, world! 36'
[INFO] [1792947267.611227734] [minimal_publisher]: Publishing: 'Hello, world! 37'
[INFO] [1792947268.111223325] [minimal_publisher]: Publishing: 'Hello, world! 38'
[INFO] [1792947268.611209149] [minimal_publisher]: Publishing: 'Hello, world! 39'
[INFO] [1792947269.111155766] [minimal_publisher]: Publishing: 'Hello, world! 40'
[INFO] [1792947269.611133789] [minimal_publisher]: Publishing: 'Hello, world! 41'
[INFO] [1792947270.111226901] [minimal_publisher]: Publishing: 'Hello, world! 42'
[INFO] [1792947270.611133672] [minimal_publisher]: Publishing: 'Hello, world! 43'
[INFO] [1792947271.111087839] [minimal_publisher]: Publishing: 'Hello, world! 44'
[INFO] [1792947271.611122325] [minimal_publisher]: Publishing: 'Hello, world! 45'
[INFO] [1792947272.111138275] [minimal_publisher]: Publishing: 'Hello, world! 46'
[INFO] [1792947272.611114149] [minimal_publisher]: Publishing: 'Hello, world! 47'
[INFO] [1792947273.111122187] [minimal_publisher]: Publishing: 'Hello, world! 48'
[INFO] [1792947273.611109300] [minimal_publisher]: Publishing: 'Hello, world! 49'
[INFO] [1792947274.111112450] [minimal_publisher]: Publishing: 'Hello, world! 50'
[INFO] [1792947274.611201019] [minimal_publisher]: Publishing: 'Hello, world! 51'
[INFO] [1792947275.111184096] [minimal_publisher]: Publishing: 'Hello, world! 52'
[INFO] [1792947275.611232217] [minimal_publisher]: Publishing: 'Hello, world! 53'
[INFO] [1792947276.111161222] [minimal_publisher]: Publishing: 'Hello, world! 54'
[INFO] [1792947276.611131220] [minimal_publisher]: Publishing: 'Hello, world! 55'
[INFO] [1792947277.111289118] [minimal_publisher]: Publishing: 'Hello, world! 56'
[INFO] [1792947277.611094696] [minimal_publisher]: Publishing: 'Hello, world! 57'
[INFO] [1792947278.111334367] [minimal_publisher]: Publishing: 'Hello, world! 58'
[INFO] [1792947278.611275975] [minimal_publisher]: Publishing: 'Hello, world! 59'
[INFO] [1792947279.111281748] [minimal_publisher]: Publishing: 'Hello, world! 60'
Distrobox: ros2-jazzy
```

Figure 25: Execution of the custom C++ subscriber node logging incoming messages.

```
tarek@asw:~/ros2_ws/src/cpp_pubsub$ cat package.xml
<?xml version="1.0"?>
<?xml-model href="http://download.ros.org/schema/package_format3.xsd" schematypens="http://www.w3.org/2001/XMLSchema"?>
<package format="3">
  <name>cpp_pubsub</name>
  <version>0.0.0</version>
  <description>TODO: Package description</description>
  <maintainer email="tarek@todo.todo">tarek</maintainer>
  <license>Apache-2.0</license>

  <buildtool_depend>ament_cmake</buildtool_depend>
  <depend>rclcpp</depend>
  <depend>std_msgs</depend>

  <test_depend>ament_lint_auto</test_depend>
  <test_depend>ament_lint_common</test_depend>

  <export>
    <build_type>ament_cmake</build_type>
  </export>
</package>
tarek@asw:~/ros2_ws/src/cpp_pubsub$ cat CMakeLists.txt
cmake_minimum_required(VERSION 3.8)
project(cpp_pubsub)

if(CMAKE_COMPILER_IS_GNUCXX OR CMAKE_CXX_COMPILER_ID MATCHES "Clang")
  add_compile_options(-Wall -Wextra -Wpedantic)
endif()

find_package(ament_cmake REQUIRED)
find_package(rclcpp REQUIRED)
find_package(std_msgs REQUIRED)

add_executable(talker src/publisher_member_function.cpp)
ament_target_dependencies(talker rclcpp std_msgs)

add_executable(listener src/subscriber_member_function.cpp)
ament_target_dependencies(listener rclcpp std_msgs)

install(TARGETS
  talker
  listener
  DESTINATION lib/${PROJECT_NAME}
)

ament_package()
tarek@asw:~/ros2_ws/src/cpp_pubsub$
```

Figure 26: Content configuration of `package.xml` and `CMakeLists.txt` for building `cpp_pubsub`.

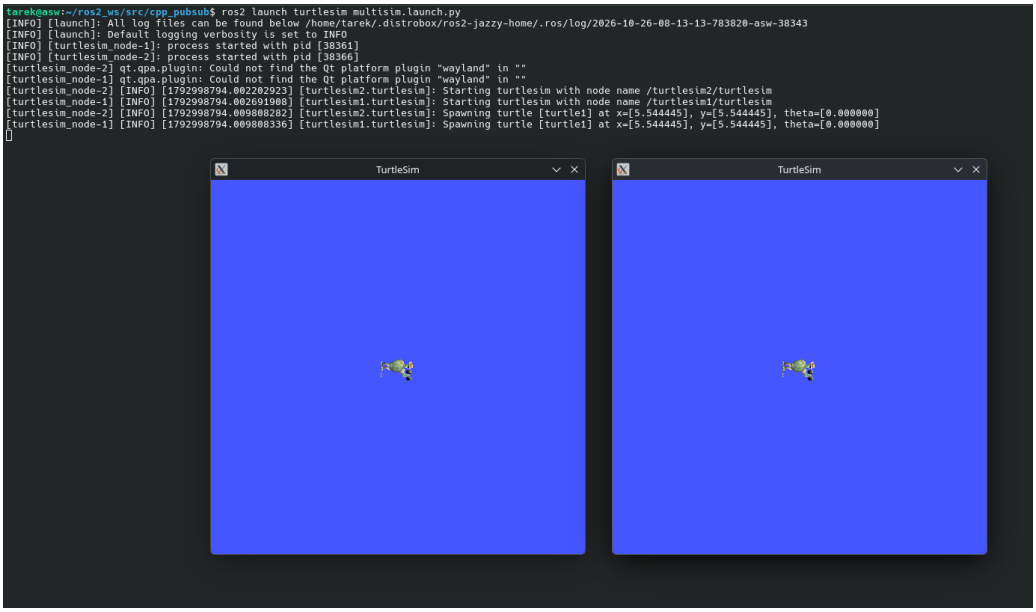


Figure 27: Launching multiple namespaced `turtlesim` instances simultaneously using `multisim.launch.py`.

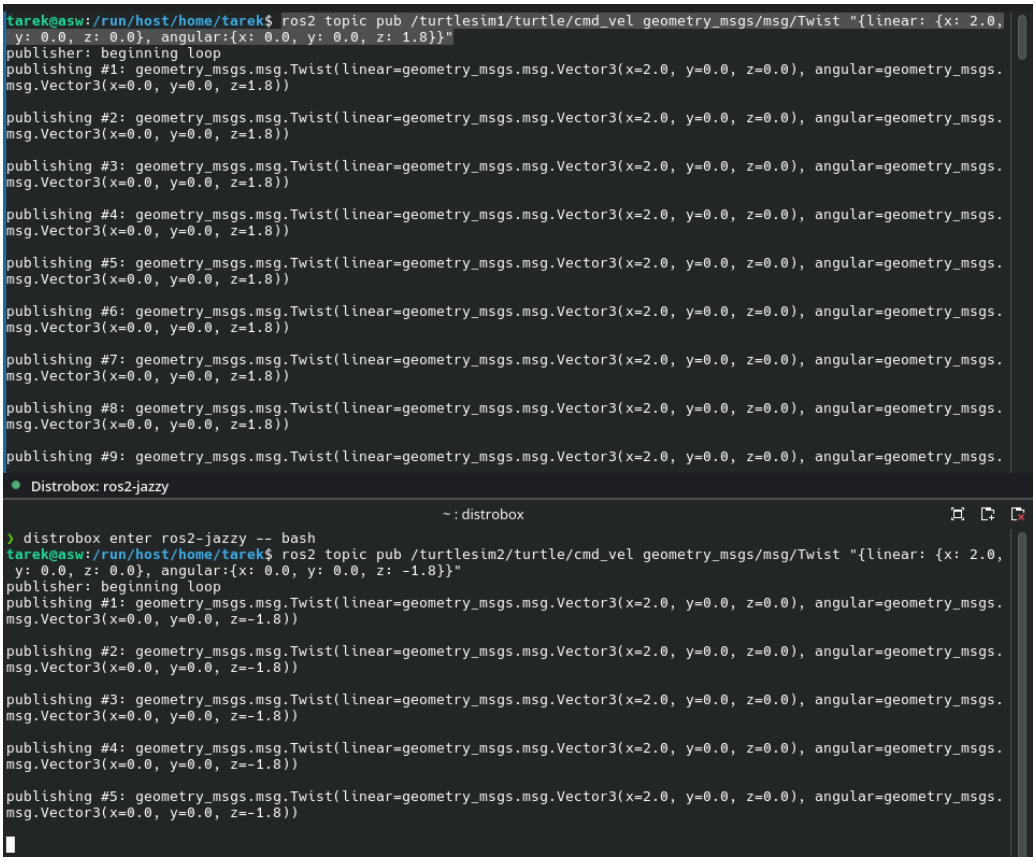


Figure 28: Publishing distinct velocity commands simultaneously to `/turtlesim1` and `/turtlesim2`.

6.2 Telemetry Recording and Replay (`rosbag2`)

Telemetry data was recorded to disk using `rosbag2`:

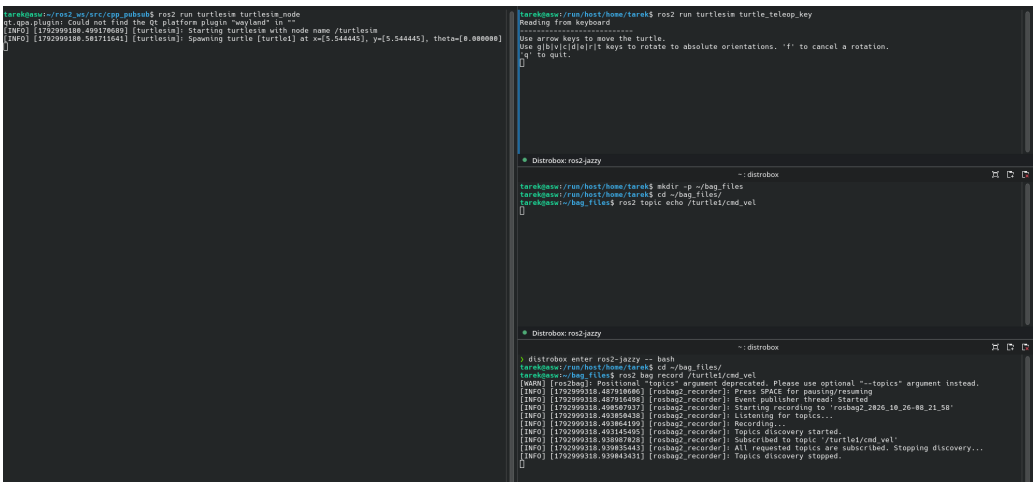


Figure 29: Recording a single topic stream (`/turtle1/cmd_vel`) using `ros2 bag record`.

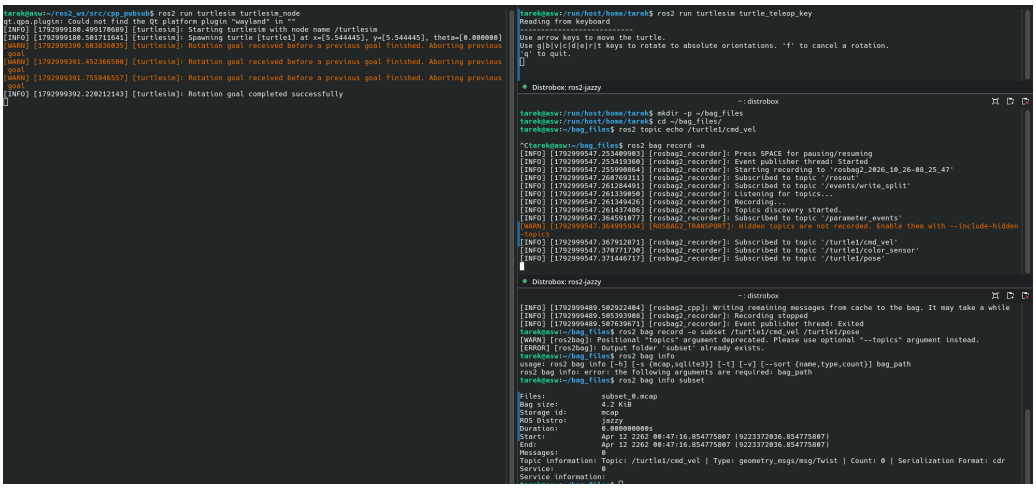


Figure 30: Recording multiple concurrent telemetry topics into a structured bag storage directory.

6.3 Visual Introspection with Rqt

Graphical diagnostics and logging inspection were analyzed through `rqt`:

7 Technical Discussion and Questions

1. **When should a Service be selected over a Topic?**
   Topics are selected for unidirectional, continuous data streams (such as odometry, velocity commands, and sensor point clouds) where latency is critical and dropped messages do not compromise overall system integrity. Services are selected for synchronous, transactional request-response interactions where receipt and verification are strictly required (such as motor resets, map saving, or coordinate spawning).

2. **What is the role of `AMENT_PREFIX_PATH`?**
   `AMENT_PREFIX_PATH` is an environment variable specifying the directory paths where ROS

```
tarek@asw:~/bag_files$ ros2 bag play subset
[WARN] [1792999621.926995470] [rosbag2_storage_mcap]: attempted to read in receive timestamp order with no message i
ndex
[INFO] [1792999621.927025732] [rosbag2_player]: Set rate to 1
[INFO] [1792999621.929970538] [rosbag2_player]: Adding keyboard callbacks.
[INFO] [1792999621.929984404] [rosbag2_player]: Press SPACE for Pause/Resume
[INFO] [1792999621.929988338] [rosbag2_player]: Press CURSOR_RIGHT for Play Next Message
[INFO] [1792999621.929992077] [rosbag2_player]: Press CURSOR_UP for Increase Rate 10%
[INFO] [1792999621.929995214] [rosbag2_player]: Press CURSOR_DOWN for Decrease Rate 10%
[INFO] [1792999621.931139396] [rosbag2_player]: Playback until timestamp: -1
tarek@asw:~/bag_files$
```

Figure 31: Replaying recorded topic streams with `ros2 bag play` to reproduce robot motion.

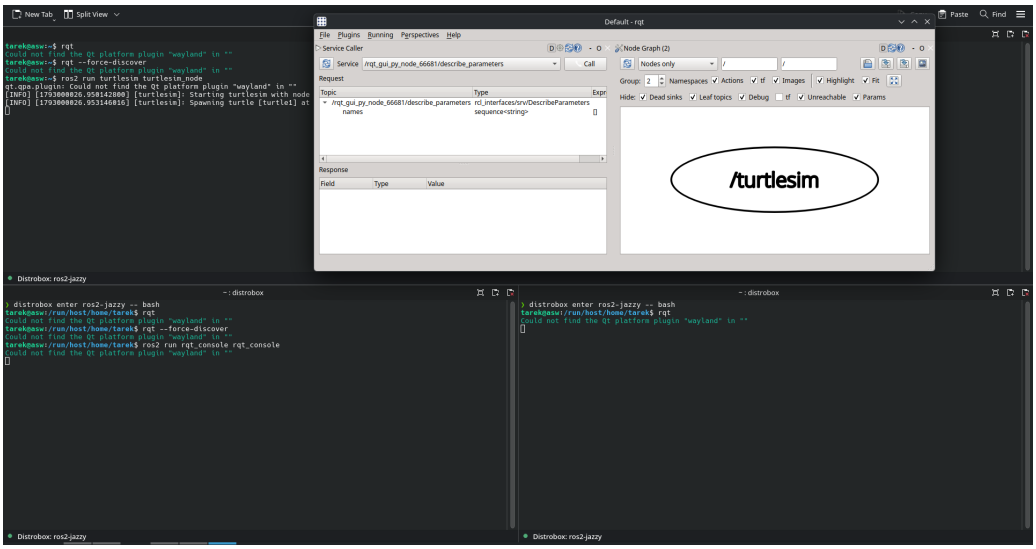


Figure 32: Introspection and log message monitoring using the `rqt` GUI suite.

2 locates package manifests, libraries, and executables. Sourcing a workspace prepends its `install/` space to the variable, establishing overlay priority over underlay distributions.

3. **What are the functions of the Colcon workspace directories?**

   - `build/`: Contains intermediate CMake caches and compiled object files.
   - `install/`: Holds final installed binaries, library targets, and environment hooks.
   - `log/`: Records structured log outputs from each package compilation step.
   - `src/`: Houses source packages, manifests, and build configuration files.

8 Conclusion

This lab covered practical interaction with the ROS 2 Jazzy computational graph. Node introspection, topic streaming, service invocation, and dynamic parameter updates were verified in practice. The mechanics of workspace overlay prioritization and custom C++ package development were demonstrated successfully within a containerized Distrobox environment on Arch Linux.